

Quebrando um cofre em Flutter: engenharia reversa e otimização de força bruta

Desafio *Fluffy* (Google CTF 2025) — Dart AOT, token derivado de timestamp e periodicidade de 16 rodadas

Integrantes do grupo

822070	Nicolas Queiroz Bertozzo
821402	Gustavo Cesar Bento Laurindo
811537	Danilo da Silva Pinto
811943	Lucas de Oliveira Rodrigues Alves
813583	Lucas de Araújo Cardoso
823127	Nicole Correa Ramos
811649	Gabriel Vianna de Oliveira Spolon
812110	Pedro Guilherme Torres das Neves
832627	Lucas Martinez Domingues

Universidade Federal de São Carlos

2 de outubro de 2026

- ① O desafio e a motivação: um cofre que só encripta
- ② Fundamentos: onde mora o código em um app Flutter (Dart AOT)
- ③ Reconstrução: âncoras de string e o algoritmo (token + cifra)
- ④ As fragilidades: token determinístico e período de 16 rodadas
- ⑤ O ataque otimizado e o custo computacional
- ⑥ Resultados: os três segredos e a flag
- ⑦ Lições aprendidas e referências

O desafio e a motivação

Por que este desafio?

- App *Flutter* que **só encripta** (decriptar está “em construção”), com algoritmo próprio (*security by obscurity*).
- Encripta com um token e um PIN de 4 dígitos.

Os três segredos (Base62):

```
fmMf7mIMbHcPoQmLGx1C00XVGBmhjTaYhB0  
506WRgCajs3QSTyohnu2hldds18mjkk  
fgv99d0vazsvEESh7DPKbb3k0I3RW
```

Nosso objetivo

- **Localizar e reconstruir** o algoritmo dentro do binário compilado (`libapp.so`, Dart AOT).
- Recuperar os três segredos *sem* conhecer o PIN nem o token.
- Demonstrar a **redução da busca** (periodicidade + horários), e não apenas força bruta “com muitos núcleos”.

Pista

1º segredo exhibe os **segundos**; “amigo da Suíça” $\Rightarrow$ fuso Europe/Zurich.

Fundamentos: de Dart a libapp.so

- Flutter *debug* usa **JIT** (interpretado, *hot reload*); Flutter *release* usa **AOT**: o código Dart é compilado para **código de máquina nativo** e embarcado num objeto ELF — o libapp.so.
- O libapp.so contém: (1) o código de todas as funções; (2) a *snapshot* (`_kDartIsolateSnapshotData`) com o *object pool*: strings, constantes e metadados; (3) tabelas do GC.
- **Por que é difícil de reverter:**
 - **Sem tabela de símbolos** das funções do app;
 - convenção de chamada e *garbage collector* próprios do Dart;
 - formato da *snapshot* **versionado** (Dart 3.8.1, snapshot 830f4f59...).
- **Mas:** strings e constantes ficam **legíveis** na seção de dados — é o ponto de entrada da análise.

Reconstrução: âncoras de string e o algoritmo

Âncoras (via strings)

gctf25_ seed do token
0123...xyz alfabeto Base62
Invalid character dica no decode

- Seguindo as *cross-references* (Ghidra/radare2) localizam-se rol8/ror8 e o laço de pin rodadas.
- blutter/darter automatizam, desde que compatíveis com o Dart 3.8.1.

Validação (round-trip)

Encriptar com token=I6X6vyQzRuH,
pin=8126 reproduz **byte a byte** o ciphertext
do 1º segredo $\Rightarrow$ reconstrução correta.

Token (derivado do tempo):

```
token = Base62( SHA1("gctf25_" + ts)[:8] )
```

Cifra (rodada i , byte j ; s =segredo, t =token):

$$s_j = \text{rol8}((s_j + t_{j \bmod 8}) \bmod 256, j \bmod 8)$$

depois: rota o array de s (direita) e de t (esquerda), e

$$t_j = \text{ror8}(t_j, (\text{pin} \oplus ((i \bmod 4) + 1)) \bmod 8)$$

O token que não é aleatório

```
token = Base62( SHA1("gctf25_" + unix_seconds)[:8] )
```

- É uma **KDF determinística do timestamp**: o “aleatório” é, na verdade, o relógio. Há só 86400 tokens por dia.
- O 1º segredo mostra 04/08/2023 13:37 **com segundos**; os demais mostram só o minuto ⇒ **60** candidatos.
- A dica “Suíça” fixa o fuso Europe/Zurich para reconstruir o timestamp (a data exibida é local; o app usa o epoch UTC).

minuto exibido	→	60 tokens candidatos
fuso Zurique	→	converte data local em epoch

A cifra: rodadas e rotações

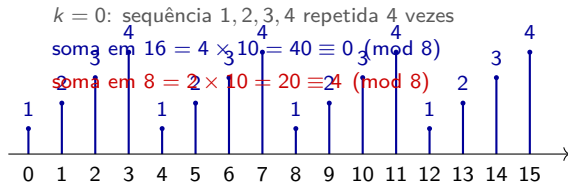
Essência do encrypt (Python equivalente ao Dart reconstruído):

```
def encrypt(pin, token, secret):          # 'pin' rodadas no total
    tok = list(base62_decode(token))
    enc = list(secret)
    for i in range(pin):
        enc = [rol8((s + tok[j % 8]) % 256, j % 8)    # soma + rotacao
                for j, s in enumerate(enc)]
        enc = [enc[-1]] + enc[:-1]                # array do segredo: roda -> direita
        tok = tok[1:] + [tok[0]]                  # array do token: roda -> esquerda
        tok = [ror8(d, (pin ^ ((i & 3) + 1)) % 8) for d in tok]
    return base62_encode(bytes(enc))
```

- O número de rodadas é o **próprio PIN** (1 a 9999) — por isso a força bruta ingênua é cara.
- Todas as operações são bijetivas $\Rightarrow$ existe decriptação (inverso em ordem reversa).
- A rotação de bits **só depende de $\text{pin} \bmod 8$ e de $i \bmod 4$.**

O período de 16 rodadas

Com $k = pin \bmod 8$, a rotação de bits é $r_i = (k \oplus ((i \bmod 4) + 1)) \bmod 8$ — período 4 em i .



$k = pin \bmod 8$	soma em 16	soma em 8
0, 1, 2, 3	$40 \equiv 0$	$20 \equiv 4$
4, 5, 6, 7	$72 \equiv 0$	$36 \equiv 4$

- Estado do token = (posição do array, rotação acumulada S_i).
- Repete quando $i \equiv 0 \pmod{8}$ e $S_i \equiv 0 \pmod{8}$.
- Em $i = 8$: $S_8 \equiv 4$ (não repete); em $i = 16$: $S_{16} \equiv 0$ (repete).

Conclusão

O token dinâmico tem período exato 16 — só 16 estados por agenda.

Conjuntos de PIN mod 8

Como r_i só depende de $k = \text{pin} \bmod 8$, há apenas **8 agendas** de rotação. Gerando os 16 estados de cada agenda e agrupando os idênticos:

- **40 estados** distintos de token no total;
- **8 estados** alcançáveis sob *todos* os 8 valores de k ;
- **32 estados** alcançáveis sob *exatamente* 2 valores de k .

	estados	pares (estado, agenda)
universais	8	$8 \times 8 = 64$
restritos	32	$32 \times 2 = 64$
total	40	128 (não $40 \times 8 = 320$)

Por que importa

Ao decriptar de trás para frente não sabemos o estado final do token nem o $\text{pin} \bmod 8$ — mas há só **128** combinações relevantes a testar.

O ataque otimizado: caminhar de trás para frente

```
for tok_pat, pin_mods in tok_patterns:    # <= 40 estados
    finais
    for pin_mod in pin_mods:              # agendas (8 ou 2)
        dyn, secret = tok_pat[:, list(base62_decode(incr))
        for step in range(max_pin):       # volta 1 rodada/
            vez
            secret = secret[1:] + [secret[0]]
            secret = [(ror8(e, j % 8) - dyn[j % 8]) % 256
                       for j, e in enumerate(secret)]
            dyn = [rol8(d, INV_ROT[pin_mod][step % 4])
                   for d in dyn]
            dyn = [dyn[-1]] + dyn[:-1]
            if is_printable(secret):       # oraculo: ASCII 0
                x20-0x7E
                return bytes(secret)
```

- Pré-computa os 40 estados finais (período 16) e, para cada par (*estado*, *agenda*), desfaz uma rodada por passo.
- A tabela INV_ROT inverte a rotação de bits em $O(1)$.
- **Oráculo:** o primeiro estado todo imprimível (0x20-0x7E) é o texto claro; o nº de passos é o PIN.

Custo por token:

$128 \times 9999 \approx 1,28M$ passos, contra
 $\sum_{p=1}^{9999} p \approx 50M$ da ingênua.

Custo computacional comparado

Estratégia	por token	total ($\times 60$)	fator
Ingênua (recalcula p rodadas por PIN)	$\approx 50M$	$\approx 3,0 \times 10^9$	$1\times$
Período 16 + PIN mod 8	$\approx 1,28M$	$\approx 7,7 \times 10^7$	$\approx 39\times$

- A redução **não** muda o nº de combinações de chave (60×9999); elimina o **trabalho redundante** de recomputar p rodadas a cada PIN.
- É exatamente a pergunta da flag: *otimizou, ou só usou muitos núcleos?* — a resposta técnica é a primeira.

Medido (Ryzen 7 5700X3D, 3 processos): **378 s, 394 s, 535 s** por segredo.

Resultados: os três segredos e a flag

Data	segundo	PIN	token	texto claro
04/08/2023 13:37	27	8126	I6X6vyQzRuH	CTF{0k4y_h4v3_u_Optimized_
07/09/2024 18:52	31	5178	BMiZFI8Xr3Q	brUt3_f0rcE_0R_y0u_jUst
03/03/2025 22:07	48	7490	KZ95PpF1RFq	_uSeD_a_l0t_of_c0Res?}

Flag completa

CTF{0k4y_h4v3_u_Optimized_brUt3_f0rcE_0R_y0u_jUst_uSeD_a_l0t_of_c0Res?}

- O 3º segredo é o mais caro: o segundo correto (48) aparece tarde na varredura dos 60.

Demo — python fluffy.py demo:

```
$ python fluffy.py demo
token      : I6X6vyQzRuH
PIN        : 8126
ciphertext : fmMf7mIMbHcPoQmLGx1C00XVGBmhjTaYhB0
round-trip : b'CTF{0k4y_h4v3_u_Optimized_}'
round-trip OK
ciphertext reproduzido: OK
```

Ataque completo (60 s varridos):

```
segredo 1 : 378 s
segredo 2 : 394 s
segredo 3 : 535 s
```

O crack otimizado, com o token correto em mãos, recupera o texto do 1º segredo em **6,5 s**. O resto do tempo é a varredura dos 60 segundos.

Como reproduzir

```
$ python fluffy.py demo
$ python fluffy.py token '04/08/2023 13:37' 27
    # -> I6X6vyQzRuH
$ python fluffy.py crack \
    '04/08/2023 13:37' \
    'fmMf7mIMbHcPoQmLGx1C00XVGBmhjTaYhB0'
    # ataque otimizado (~6 min)
```

- fluffy.py é autocontido (só biblioteca padrão): token, Base62, cifra e o crack otimizado.
- crack.py é o solver oficial (mesma lógica).
- O relatório completo está em [relatorio-engenharia-reversa](#)

- **Security by obscurity não é segurança:** num binário sem símbolos, as strings entregaram o algoritmo inteiro.
- **Chave derivada do tempo é chave fraca:** o “token aleatório” era o relógio — entropia real (64 bits sorteados) é essencial.
- **Estrutura periódica vaza:** o período de 16 rodadas permitiu reaproveitar o ciclo e cortar o custo em $\approx 39\times$ — projetar cifra exige pensar na repetição de estado.
- **RE e criptoanálise juntas:** localizar o algoritmo (RE) foi metade; entender a matemática da cifra (criptoanálise) foi a outra.

- Google CTF 2025 — *Fluffy* (repositório, fonte e write-up):
<https://github.com/google/google-ctf/tree/1655538e8c8b41451d39f670ef15a5af22979ca9/2025/quals/rev-fluffy>
- blutter (reversão de Flutter/Dart AOT): <https://github.com/worawit/blutter>
- darter (parser de snapshot Dart): <https://github.com/mildsunrise/darter>
- frida (instrumentação dinâmica): <https://frida.re>
- Documentação Dart AOT (gen_snapshot): <https://dart.dev>

Obrigado!

Perguntas?